Handoff: microarquitectura RV32I multiciclo (MT-7003)

Oct 24, 2026 · @Felipe

Contexto

Estudiante del ITCR (Costa Rica) que implementa una microarquitectura RV32I en Verilog y Vivado para el curso MT-7003 Microprocesadores y microcontroladores (Dr. Carlos Salazar García, II semestre 2026, 20 % de la nota). La persona se define como floja en código y necesita entender todo para defenderlo: si usa IA y no puede defenderlo ante el profesor, la nota es la mínima. Por eso el trabajo es guiado, entregar código hecho comentado para que lo comprenda

Entregas del enunciado

- **Avance 1 (semana 11):** explicación breve de qué instrucciones RV32I debe soportar la arquitectura (sacadas de compilar el código C del profesor) y circuito propuesto para tipos I y S, sin diagramas a mano. También se verifica el proceso de compilar y extraer el binario o hexadecimal. Ya se presentó, y el profesor pidió corregir: la arquitectura debe ser **multiciclo**.
- **Avance 2 (semana 13):** circuitos de todas las decodificaciones y la implementación que se tenga hasta ese momento.
- **Entrega final (semana 17):** microarquitectura completa, informe de máximo 6 páginas en formato IEEE de dos columnas, bitácora y revisión.

Reglas técnicas

- El módulo principal solo tiene dos entradas: Clock y Reset.
- Durante la evaluación solo se puede cambiar la memoria de instrucciones.
- Cada módulo se documenta (función, entradas, salidas y razón de existir) y tiene su testbench (objetivo, estímulos y resultados esperados).
- Vivado debe volcar la memoria de datos a un archivo con el formato «Dirección de Memoria / Valor en hexadecimal».
- Se pueden ignorar las instrucciones posteriores al «}» de main, las llamadas al sistema operativo y los overheads.
- Se permite usar IA, pero cada componente debe poder defenderse citando una instrucción RISC-V o un estilo de decodificación.

Estado actual

El diseño multiciclo en papel está completo y probado con un modelo en Python; lo que falta es implementarlo en Verilog, y la persona quiere construirlo desde cero con guía.

Hecho

- **Programa del profesor compilado:** 44 instrucciones en Compiler Explorer con RISC-V clang 18.1.0, ya enlazado y convertido a hexadecimal, más una tabla con dirección, hex, binario e instrucción.
- **Lienzo con cuatro láminas multiciclo** (ruta de datos, instrucciones, máquina de estados, señales de control), publicado como artefacto privado. Cada lámina está en HTML dentro del lienzo.
- **Verificación del diseño:** un modelo ciclo a ciclo en Python que usa las mismas señales de las láminas ejecutó el programa y dio r = 0xC7DF, a = 0x62, b = 2 e i = 2, con las mismas 58 instrucciones que el emulador de referencia (unos 240 ciclos).
- **Explicaciones ya dadas a la persona:** qué hace el C del profesor, qué es un char, qué se desplaza en `b << 1`, dónde viven las variables en la pila, cómo se codifica una instrucción a mano, hex frente a binario, multiciclo frente a segmentado, estados frente a instrucciones y el flujo completo de la ruta de datos.
- **Adelanto en Verilog (diseño viejo, monociclo)** con testbenches por módulo. Todos pasaron en el Vivado de la persona después de corregir la ruta del archivo hex.

Pendiente inmediato

- Empezar la implementación del Avance 2 (ver próximos pasos).

Decisiones de diseño

Cada decisión tiene una razón que la persona debe poder defender. La de la memoria única es preferencia de la persona y falta reflejarla en las láminas.

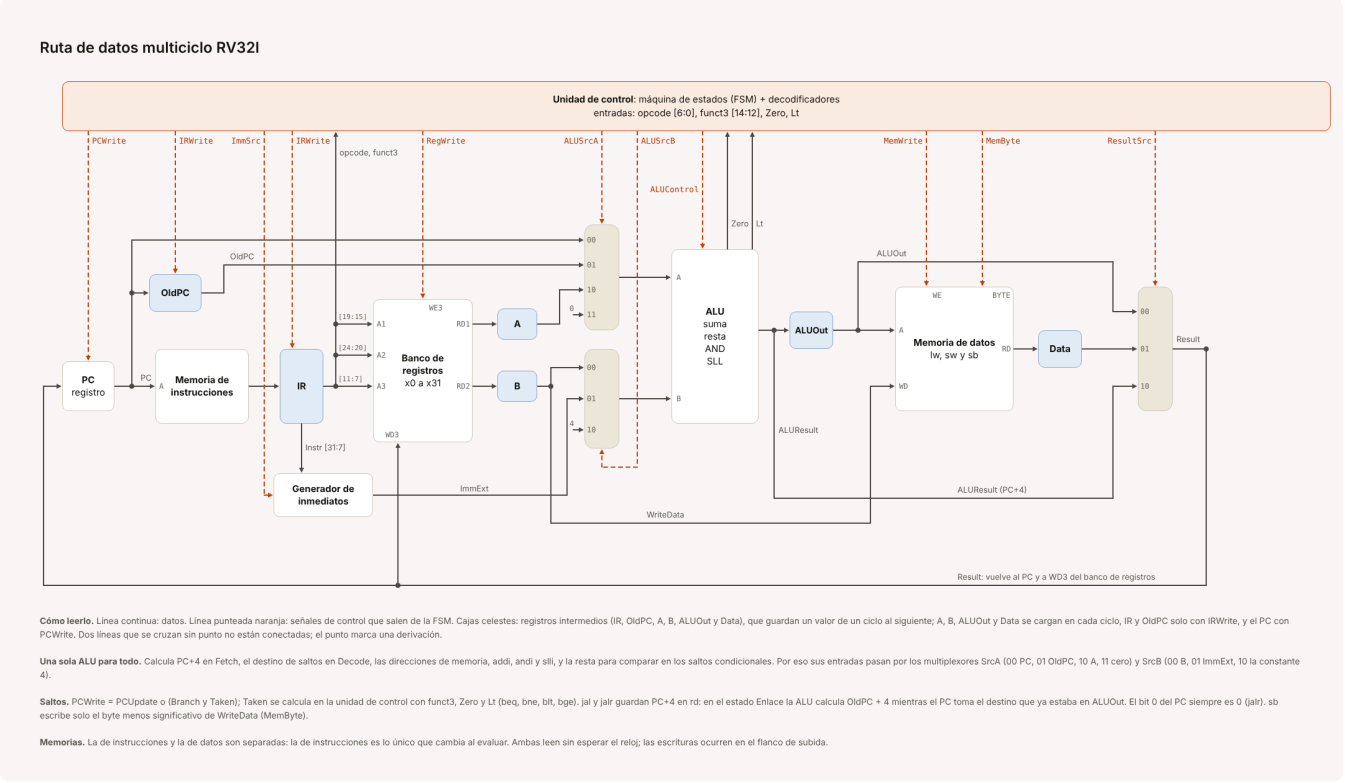
Decisión	Por qué
Arquitectura multiciclo, no segmentada	El profesor pidió cambiar a multiciclo en la revisión del Avance 1. Una instrucción se ejecuta en varios ciclos, uno por estado, y solo hay una en ejecución a la vez. La persona creía que significaba instrucciones simultáneas, que es segmentado (pipeline).

Decisión	Por qué
Memoria única para instrucciones y datos (preferencia de la persona)	Es lo que marca la teoría del multiciclo. Un multiplexor de dirección (AdrSrc: 0 = PC, 1 = ALUOut) elige qué direcciona la memoria en cada ciclo. Falta confirmar con el profesor cómo se carga el programa, porque el enunciado dice que en la evaluación solo cambia la memoria de instrucciones.
Solo las instrucciones que usa el programa	addi, andi, slli, lw, jalr, sw, sb, lui, beq, bne, blt, bge y jal. Sin tipo R, ori, xori, slti, srli, lb, lh ni sh, porque el enunciado limita el soporte a lo necesario.
Una sola ALU compartida	Suma PC+4 en Fetch, calcula destinos en Decode, direcciones, addi/andi/slli y la resta de los saltos. Por eso sus entradas pasan por los multiplexores SrcA y SrcB.
ImmSrc se decodifica solo con el opcode; MemByte con funct3	Así la FSM no necesita estados distintos por tipo de inmediato. MemByte = 1 si funct3 es 000 (sb).
Saltos con las banderas Zero y Lt	Taken = funct3[2] ? funct3[0] XOR Lt : funct3[0] XOR Zero. Cubre beq (000), bne (001), blt (100) y bge (101). Lt es A < B con signo.
jal y jalr comparten el estado Enlace (S9)	El PC toma el destino que ya está en ALUOut mientras ALUOut recibe OldPC + 4 para guardarlo en rd. El bit 0 del PC siempre es 0.
lui = 0 + inmediato tipo U	SrcA = 11 entrega cero a la ALU, así lui reutiliza el estado de escritura S7.
Registros de los saltos: OldPC	El PC ya avanzó cuando se calcula el destino, y los saltos son relativos a la dirección de su propia instrucción.
Compilar con clang 18.1.0 (RISC-V rv32gc) en Compiler Explorer	No hay GCC disponible ahí. El profesor podría usar GCC en la entrega final y escoger variantes (ble en vez de blt, mv en vez de li), pero las familias de instrucciones son las mismas.

Decisión	Por qué
Hexadecimal con $readmemh	La memoria de instrucciones lee un archivo con una palabra de 8 dígitos hex por línea; el volcado de la memoria de datos también va en hexadecimal.

Arquitectura multiciclo

Una sola ALU y una máquina de estados de 12 estados ejecutan cada instrucción en 3 a 5 ciclos de reloj, y los registros intermedios guardan el resultado de cada ciclo para el siguiente. Las imágenes son capturas de las láminas del lienzo.



Cómo leerlo. Línea continua: datos. Línea punteada naranja: señales de control que salen de la FSM. Cajas celestes: registros intermedios (IR, OldPC, A, B, ALUOut y Data), que guardan un valor de un ciclo al siguiente; A, B, ALUOut y Data se cargan en cada ciclo, IR y OldPC solo con IRWrite, y el PC con PCWrite. Dos líneas que se cruzan sin punto no están conectadas; el punto marca una derivación.

Una sola ALU para todo. Calcula PC+4 en Fetch, el destino de saltos en Decode, las direcciones de memoria, addi, andi y slli, y la resta para comparar en los saltos condicionales. Por eso sus entradas pasan por los multiplexores SrcA (00 PC, 01 OldPC, 10 A, 11 cero) y SrcB (00 B, 01 ImmExt, 10 la constante 4).

Saltos. PCWrite = PCUpdate o (Branch y Taken); Taken se calcula en la unidad de control con funct3, Zero y Lt (beq, bne, blt, bge). jal y jalr guardan PC+4 en rd: en el estado Enlace la ALU calcula OldPC + 4 mientras el PC toma el destino que ya estaba en ALUOut. El bit 0 del PC siempre es 0 (jalr). sb escribe solo el byte menos significativo de WriteData (MemByte).

Memorias. La de instrucciones y la de datos son separadas: la de instrucciones es lo único que cambia al evaluar. Ambas leen sin esperar el reloj; las escrituras ocurren en el flanco de subida.

La imagen todavía muestra dos memorias. Con memoria única se agrega un multiplexor AdrSrc antes de la dirección (0 = PC, 1 = ALUOut), y una sola salida de lectura alimenta a IR y a Data. AdrSrc vale 0 en S0 y 1 en S3 y S5; en los demás estados no importa.

Componentes y registros

Componente	Qué hace
PC	Registro con enable. PCWrite = PCUpdate o (Branch y Taken). El bit 0 siempre es 0.

Componente	Qué hace
IR y OldPC	Guardan la instrucción y su dirección; se cargan solo con IRWrite (estado S0).
A y B	Copian RD1 y RD2 del banco de registros en cada ciclo.
ALUOut	Copia el resultado de la ALU en cada ciclo.
Data	Copia lo leído de la memoria de datos en cada ciclo.
Mux SrcA	00 PC, 01 OldPC, 10 A, 11 cero.
Mux SrcB	00 B, 01 ImmExt, 10 la constante 4.
Mux de resultado	00 ALUOut, 01 Data, 10 ALUResult directo. Su salida va al PC y a WD3 del banco de registros.
ALU	ALUControl 000 suma, 001 resta, 010 AND, 110 SLL. Entrega Zero y Lt a la unidad de control.
Memoria de datos	Dirección = ALUOut, dato de escritura = B. MemWrite escribe; MemByte limita la escritura al byte menos significativo (sb).

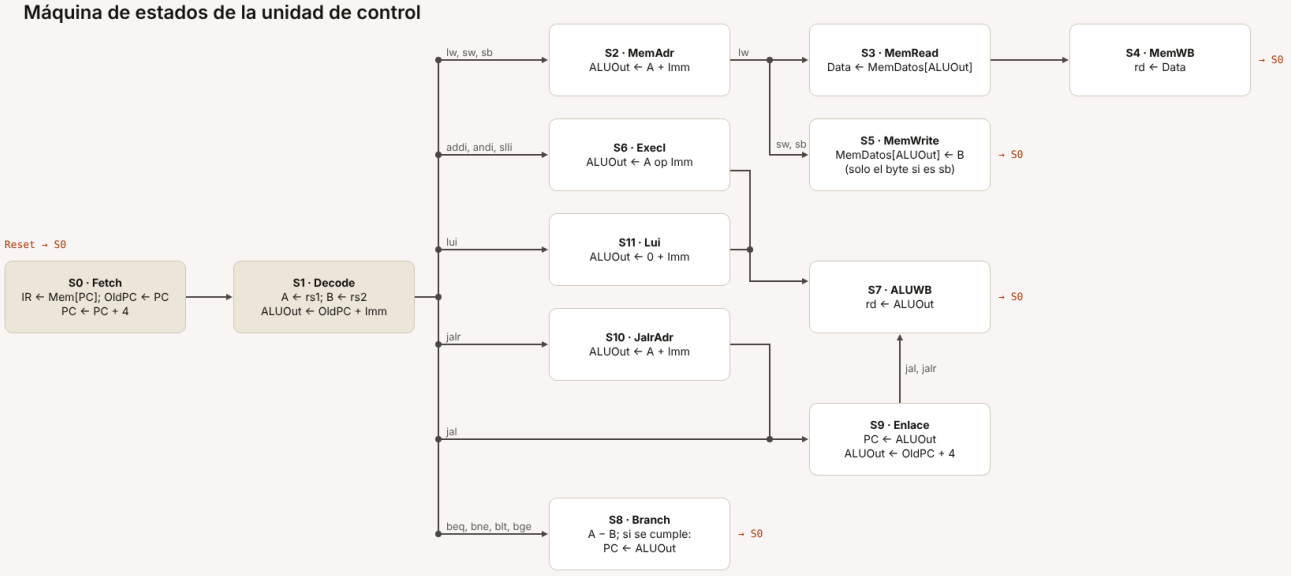
Señales por estado. X significa que no importa (en Verilog se escribe 0). ALUOp: 00 suma, 01 resta, 10 según funct3.

Estado	Qué hace	PCUpdate	Branch	IRWrite	RegWrite	MemWrite	ALUSrcA	ALUSrcB	ALUOp	ResultSrc
S0 Fetch	IR ← Mem[PC]; OldPC ← PC; PC ← PC + 4	1	0	1	0	0	00	10	00	10
S1 Decode	A ← rs1; B ← rs2; ALUOut ← OldPC + Imm	0	0	0	0	0	01	01	00	X
S2 MemAdr	ALUOut ← A + Imm	0	0	0	0	0	10	01	00	X
S3 MemRead	Data ← MemDatos[ALUOut]	0	0	0	0	0	X	X	X	X
S4 MemWB	rd ← Data	0	0	0	1	0	X	X	X	01
S5 MemWrite	MemDatos[ALUOut] ← B	0	0	0	0	1	X	X	X	X
S6 Execl	ALUOut ← A op Imm	0	0	0	0	0	10	01	10	X
S7 ALUWB	rd ← ALUOut	0	0	0	1	0	X	X	X	00
S8 Branch	A − B; si se cumple, PC ← ALUOut	0	1	0	0	0	10	00	01	00
S9 Enlace	PC ← ALUOut; ALUOut ← OldPC + 4	1	0	0	0	0	01	10	00	00
S10 JalrAdr	ALUOut ← A + Imm	0	0	0	0	0	10	01	00	X
S11 Lui	ALUOut ← 0 + Imm	0	0	0	0	0	11	01	00	X

Decodificadores y siguiente estado

- **ImmSrc, solo con el opcode:** 0010011, 0000011 y 1100111 dan tipo I (000); 0100011 tipo S (001); 1100011 tipo B (010); 1101111 tipo J (011); 0110111 tipo U (100).
- **ALU:** ALUOp 00 da suma; 01 da resta; 10 usa funct3, con 000 suma (addi), 111 AND (andi) y 001 SLL (slli).
- **Siguiente estado:** S0 va a S1. S1 va a S2 con lw y sw/sb (opcodes 0000011 y 0100011), a S6 con 0010011, a S11 con 0110111, a S8 con 1100011, a S9 con 1101111 y a S10 con 1100111. S2 va a S3 si es lw y a S5 si es sw/sb. S3 va a S4. S6, S11 y S9 van a S7. S10 va a S9. S4, S5, S7 y S8 vuelven a S0.

Máquina de estados de la unidad de control



Cada estado dura un ciclo de reloj. Fetch y Decode son iguales para toda instrucción; desde Decode el opcode decide el camino. Todos los caminos terminan volviendo a S0, que busca la siguiente instrucción.

Instrucción	Estados que recorre	Ciclos
addi, andi, slli	S0 → S1 → S6 → S7	4
lui	S0 → S1 → S11 → S7	4
lw	S0 → S1 → S2 → S3 → S4	5
sw, sb	S0 → S1 → S2 → S5	4
beq, bne, blt, bge	S0 → S1 → S8	3
jal	S0 → S1 → S9 → S7	4
jalr	S0 → S1 → S10 → S9 → S7	5

Ejemplo, lw a1, -0x1c(s0): S0 trae la instrucción; S1 lee s0 en A; S2 suma A + (−28) y deja la dirección en ALUOut; S3 lee la memoria de datos hacia Data; S4 escribe Data en a1. Son 5 ciclos.

Ciclos por instrucción

Instrucción	Estados que recorre	Ciclos
addi, andi, slli	S0, S1, S6, S7	4
lui	S0, S1, S11, S7	4
lw	S0, S1, S2, S3, S4	5

Instrucción	Estados que recorre	Ciclos
sw, sb	S0, S1, S2, S5	4
beq, bne, blt, bge	S0, S1, S8	3
jal	S0, S1, S9, S7	4
jalr	S0, S1, S10, S9, S7	5

Programa del profesor y su compilación

El programa (pm_ev_grupos_6_estudiantes.c) deja r = 0xC7DF, a = 0x62, b = 2 e i = 2, y compilado ocupa 44 instrucciones.

```
int main(){
  int *r = 0xC7DF;   // nunca se dereferencia
  char a = 'c';      // 99
  int b = 33;
  for (int i=0; i<2; i++) {
    if (i==0){ a = 'b'; }          // 98
    else{ b = b<<1; b = b&0xF; }   // 33 -> 66 -> 2
  }
}
```

El original incluye stdio.h y un printf comentado; para compilar en Compiler Explorer se quitó el include y se agregó `-Wno-error=int-conversion` porque `int *r = 0xC7DF` es una conversión entera a puntero.

Cómo se compiló

- Compiler Explorer (godbolt.org), lenguaje C, compilador RISC-V rv32gc clang 18.1.0, opciones `-march=rv32i -mabi=ilp32 -O0 -Wno-error=int-conversion`. No existe GCC en esa lista.
- Sin `-ffreestanding`: esa opción daba 43 instrucciones y se descartó.
- Compiler Explorer muestra el objeto sin enlazar (los `j` aparecen con relocaciones R_RISCV_JAL) y su hex no se puede copiar como texto. Por eso el hex final se generó enlazando en la dirección 0 con ld.lld y extrayendo con llvm-objcopy. Con `-O0` las variables viven en la pila.
- El profesor pide que se pueda mostrar este proceso en el Avance 1, así que la persona debe poder explicar cada paso.

Programa en ensamblador (dirección, instrucción)

```
0x00 addi sp, sp, -0x20      0x3c lw a1, -0x1c(s0)       0x78 lw a0,
-0x18(s0)
0x04 sw ra, 0x1c(sp)         0x40 li a0, 0x1             0x7c andi a0, a0,
0xf
0x08 sw s0, 0x18(sp)         0x44 bge a0, a1, 0x4c       0x80 sw a0,
-0x18(s0)
0x0c addi s0, sp, 0x20       0x48 j 0x9c                 0x84 j 0x88
0x10 li a0, 0x0              0x4c j 0x50                 0x88 j 0x8c
0x14 sw a0, -0xc(s0)         0x50 lw a0, -0x1c(s0)       0x8c lw a0,
-0x1c(s0)
0x18 lui a1, 0xc             0x54 beqz a0, 0x5c          0x90 addi a0, a0,
0x1
0x1c addi a1, a1, 0x7df      0x58 j 0x6c                 0x94 sw a0,
-0x1c(s0)
0x20 sw a1, -0x10(s0)        0x5c j 0x60                 0x98 j 0x3c
0x24 li a1, 0x63             0x60 li a0, 0x62            0x9c lw a0, -0xc(s0)
0x28 sb a1, -0x11(s0)        0x64 sb a0, -0x11(s0)       0xa0 lw ra, 0x1c(sp)
0x2c li a1, 0x21             0x68 j 0x88                 0xa4 lw s0, 0x18(sp)
0x30 sw a1, -0x18(s0)        0x6c lw a0, -0x18(s0)       0xa8 addi sp, sp,
0x20
0x34 sw a0, -0x1c(s0)        0x70 slli a0, a0, 0x1       0xac ret
0x38 j 0x3c                  0x74 sw a0, -0x18(s0)
```

Hexadecimal enlazado (una palabra por línea, para `$readmemh`, de la dirección 0x00 a la 0xac)

```
fe010113 00112e23 00812c23 02010413 00000513 fea42a23 0000c5b7 7df58593
feb42823 06300593 feb407a3 02100593 feb42423 fea42223 0040006f fe442583
00100513 00b55463 0540006f 0040006f fe442503 00050463 0140006f 0040006f
06200513 fea407a3 0200006f fe842503 00151513 fea42423 fe842503 00f57513
fea42423 0040006f 0040006f fe442503 00150513 fea42223 fa5ff06f ff442503
01c12083 01812403 02010113 00008067
```

Datos que importan para la memoria de datos

- sp vale 0 al arrancar (se ignora el código de arranque), así que el prólogo lo deja en 0xFFFFFFE0 y s0 queda en 0. Las variables caen en direcciones 0xFFFFFFxx: r en 0xFFFFFFF0, a en 0xFFFFFFEF, b en 0xFFFFFFE8, i en 0xFFFFFFE4, ra guardado en 0xFFFFFFFC y s0 guardado en 0xFFFFFFF8.
- El programa ejecuta 58 instrucciones y, con la tabla de ciclos, unos 240 ciclos.
- El `ret` final salta a ra, que vale 0, es decir, al inicio del programa.

Archivos, enlaces y entorno

La fuente de verdad de los diagramas es el lienzo de diseño, un artefacto privado (versión 8) que la persona debe compartir desde su menú Share si quiere enseñárselo al profesor o a su grupo.

Qué	Dónde	Estado
Cuatro láminas multiciclo	Lienzo: Main.dc.html (ruta de datos), Instrucciones.dc.html, Estados.dc.html, Control.dc.html	Vigentes salvo la memoria, que muestran como dos separadas. Para cambiarlas hay que leer el archivo del lienzo y publicarlo de nuevo en la misma URL.
Hex y tabla del programa	programa_godbolt.hex y programa_godbolt_tabla.csv (dirección, hex, binario, instrucción)	Enviados a la persona. Usar el hex como `programa.hex`; descartar archivos anteriores de 43 instrucciones.
Verilog del adelanto (diseño viejo)	riscv_avance1.zip con src, tb, programs y tools	Enviado a la persona. No coincide con el diseño multiciclo.
Enunciado	Enunciado del proyecto.pdf, que la persona adjuntó	Es la referencia de reglas y entregas.
Código del profesor	pm_ev_grupos_6_estudiantes.c, que la persona adjuntó	Es el programa que debe ejecutar la arquitectura.

En la sesión anterior había además una carpeta de trabajo con el generador de las láminas (gen.py), el modelo ciclo a ciclo (fsm_model.py), un emulador RV32I independiente (ejecutar.py) y el emulador del Verilog viejo. Esa carpeta vive en un contenedor temporal y puede no existir; lo que importa está en el lienzo, en los archivos enviados y en este documento.

Entorno de la persona

- Simula en Vivado. Para `$readmemh` usa la ruta completa con barras normales (`C:/ruta/programa.hex`), porque una ruta relativa se resuelve contra la carpeta de simulación y falla.
- Edita archivos en VS Code.
- Habla español de Costa Rica y espera respuestas en español.

Verilog existente y qué hay que cambiar

El Verilog que ya existe es un adelanto del diseño monociclo viejo (solo tipos I y S) y sirve de base parcial. Todos sus testbenches pasaron en el Vivado de la persona, pero aún incluye ori, xori, slti y srli, y no tiene sb, jalr, lui, saltos ni jal.

Módulo	Qué hace hoy	Cambio para multiciclo
alu	add, sub, AND, OR, XOR, SLT, SLL y SRL (ALUControl 000 a 111)	Se reutiliza. Agregar salidas Zero y Lt (comparación con signo). Solo se usan suma, resta, AND y SLL.
regfile	32 registros, x0 siempre 0, escritura en flanco	Se reutiliza.
imm_gen	Tipos I y S (ImmSrc de 2 bits)	Ampliar a I, S, B, J y U con ImmSrc de 3 bits (000 I, 001 S, 010 B, 011 J, 100 U).
instr_mem	`$readmemh(MEM_FILE, mem)`, relleno con NOP 0x00000013	Se reutiliza.
data_mem	Lectura y escritura de palabra, tarea `dump` que escribe «Direccion de Memoria Valor en hexadecimal»	Agregar escritura de un byte (MemByte) y decidir cómo mapear las direcciones 0xFFFFFFxx de la pila.
pc_reg	Registro del PC	Pasar a registro con enable (PCWrite) y bit 0 siempre en 0.
adder y mux2	Sumador de PC+4 y multiplexor de 2 entradas	Ya no hay sumador de PC+4 aparte; mux2 sirve de base para los multiplexores de 3 y 4 entradas.
control_unit	Decodificador combinacional	Rehacer como FSM de 12 estados con los decodificadores de ImmSrc, MemByte, ALUControl y Taken.
riscv_top	Conecta todo, solo Clock y Reset	Reconectar con IR, OldPC, A, B, ALUOut y Data.
Nuevos	No existen	Registro con enable (para IR, OldPC y PC) y registros simples (A, B, ALUOut, Data), más multiplexores de 3 y 4 entradas.

Los testbenches de los módulos que se reutilizan siguen valiendo. Hay que reescribir los de control_unit y riscv_top y escribir los de los módulos nuevos. Cada testbench documenta objetivo, estímulos y resultados esperados, como exige el enunciado.

Con memoria única, instr_mem y data_mem se fusionan en un solo módulo con el multiplexor de dirección.

Próximos pasos y forma de trabajar

Lo siguiente es que la persona escriba el Verilog del Avance 2 desde cero, con revisión mía, en este orden propuesto (aún sin confirmar por ella):

1. La persona entrega los dos ejercicios pendientes: la codificación a mano de `li a0,0`, `sw a0,-12(s0)` y `lui a1,0xc`, y el trazado de `sw a1, -0x10(s0)` ciclo por ciclo.
2. Construir módulo por módulo: ALU con Zero y Lt, banco de registros, generador de inmediatos de cinco tipos, memorias, registros con enable y multiplexores, la FSM de control y al final el módulo principal. Empezar por la ALU, preguntando por qué necesita dos banderas y cuál usa cada salto.
3. Probar cada módulo con su testbench y luego correr el programa del profesor, comparando con el modelo: r = 0xC7DF, a = 0x62, b = 2, i = 2, 58 instrucciones y unos 240 ciclos.
4. Preparar el volcado de la memoria de datos y la defensa de cada componente.

Decisiones abiertas

- **Pila:** sp vale 0 al arrancar, así que las variables caen en 0xFFFFFFxx. Hay que decidir cómo se mapea a la memoria de datos (recortar direcciones, inicializar sp en el reset u otra opción) y qué direcciones aparecen en el volcado.
- **Formato del programa:** se usa hexadecimal con `$readmemh`. Preguntar al profesor si acepta binario o solo hexadecimal.
- **Memoria:** se usará una memoria única, como prefiere la persona. Faltan actualizar las láminas y confirmar con el profesor cómo se carga el programa.
- **Fin del programa:** el `ret` final salta a 0 y reiniciaría el programa. Decidir si el PC se detiene ahí o si se deja repetir sin afectar el volcado.
- **Reentrega del Avance 1:** confirmar con el profesor qué debe volver a presentar y cuándo.

Cómo trabajar con esta persona

- Responder en español, con tono cercano y explicando el porqué antes del cómo, usando ejemplos del propio programa.

- No entregar código hecho: guiar, hacer que lo escriba y revisarlo, y comprobar la comprensión con ejercicios cortos. Ella lo dijo: necesita defenderlo ante el profesor.
- Distinguir siempre instrucción de estado, y monociclo, multiciclo y segmentado, que fueron fuente de confusión.
- Los diagramas se hacen en el lienzo, nunca a mano, y se contrastan con el modelo ciclo a ciclo antes de darlos por buenos.